

CSCE 627 THEORY OF COMPUTABILITY · MODULE 08 · LECTURE NOTES

Undecidability in the Wild

The theorems, in tools you used this week.

MODULE OUTCOMES

Identify undecidable problems in compilers and type systems.

Identify them in program analysis and verification.

Explain why they appear and what the tool does instead.

Explain the Halting-adjacent problems in security.

Read a tool's documentation for its soundness position.

1 In your compiler

Question	Status	What the compiler does instead
Is this code reachable?	Undecidable	Conservative approximation: assume reachable unless provably not. Which is why dead-code elimination misses dead code.
Do these two pointers alias?	Undecidable	Assume they might, and lose the optimisation — see the callout, which is the most consequential row here.
Does this loop terminate?	Undecidable	Recognise the easy patterns only — a monotone counter against a constant bound — and give up otherwise.
Is this grammar ambiguous?	Undecidable	Report the conflicts the particular method found (Module 03 §2), which is a diagnostic rather than a verdict.
Are these two expressions equal?	Undecidable	Syntactic matching plus a fixed list of known identities — which is why common-subexpression elimination misses obvious cases.
Is this the optimal code for this program?	Undecidable	Apply a fixed sequence of rewrites and stop. Which is why -O3 is sometimes slower than -O2: the sequence is a heuristic, not an optimisation.

Every row is Rice's theorem (Module 06 §2) — these are all non-trivial semantic properties of programs — and **the pattern of response is always the same: be conservative, or handle the easy cases, or bound a resource. Which means a compiler's limitations are largely predictable from the theory**, and that is a genuinely useful thing when you are wondering why an optimisation did not fire.

ALIAS ANALYSIS IS THE MOST EXPENSIVE ONE

'Can these two pointers refer to the same memory location?' If the compiler cannot rule it out, **it must assume they can**, because assuming otherwise would change the program's meaning.

And that assumption blocks instruction reordering, caching values in registers across stores, and vectorisation — so the undecidability of aliasing costs measurable performance in every C and C++ program ever compiled.

Which is why restrict exists — a promise from the programmer that the compiler cannot verify — **and why Fortran historically outperformed C on numerical code: Fortran's aliasing rules let the compiler assume non-aliasing by default**, so it did not have to prove anything.

And it is why Rust's borrow checker is interesting in this context: it makes the programmer supply, in the type system, the proof the compiler could not find — the aliasing information becomes a checkable annotation rather than an undecidable inference. **Which is Module 12 §3's third strategy, in a shipping language**, and it is the clearest case in the program of an undecidability result shaping a language's design.

2 In your type checker

TYPE CHECKING IS DECIDABLE BECAUSE THE LANGUAGE WAS DESIGNED TO MAKE IT SO

Type inference for the full lambda calculus with polymorphic recursion is undecidable. So languages restrict what they will infer, deliberately, to stay inside a decidable fragment.

Hindley–Milner inference is decidable — DEXPTIME-complete in the worst case and effectively linear on real programs — **because let-polymorphism is restricted in precisely the way that preserves decidability. The restriction is not an oversight; it is the design.**

And dependent types push against the boundary: type checking remains decidable only if the language guarantees that type-level computation terminates — which is why Agda and Idris ship totality checkers, and why they reject some programs whose termination they cannot establish (Module 05 §4's restriction response).

So the rule is: a type checker rejects some programs that would have worked, and that is the price of always terminating with an answer — the only alternative is a checker that sometimes does not answer at all. Which reframes the familiar frustration of fighting a type checker: it is not being obtuse, it is being total.

```

SOUND      never accepts a bad program
            (no false negatives)
COMPLETE   never rejects a good program
            (no false positives)

```

RICE SAYS YOU CANNOT HAVE BOTH AND TERMINATE. Pick two of three: SOUND, COMPLETE, TERMINATING.

WHO PICKS WHAT:

```

TYPE CHECKERS      sound + terminating
-> they reject valid programs. You have met this.

```

```

MOST LINTERS        neither + terminating
-> false positives AND false negatives, and they are
    still useful because they are cheap and fast.

```

```

VERIFIERS (Astree,    sound + terminating
SPARK, bounded CBMC)
-> many false positives, BY DESIGN, because in their
    domain a missed bug is unacceptable.

```

```

BUG FINDERS (Infer,   complete-ish + terminating
Coverity)
-> they prefer not to cry wolf, so they miss bugs
    deliberately. A product decision, not a defect.

```

```

PROOF ASSISTANTS      sound + complete
-> and NOT automatically terminating. A human
    supplies the search the tool cannot.

```

'Pick two of three' is the whole design space, and knowing which two a tool picked tells you exactly what its output licenses you to believe. It is the most useful frame in this module, and it applies to every analyser you will ever use — including ones that do not describe themselves in these terms, which is most of them.

3 In verification and security

- **'Is this program malware?' Undecidable** — malicious behaviour is a semantic property, so Rice applies directly, **and Cohen proved the virus-detection version in 1987**. So detection is signatures, behavioural heuristics, and sandboxed execution, all of which are approximations with both error types.
- **'Does this program leak secrets?'** Non-interference — that public outputs do not depend on secret inputs — is undecidable in general; practical systems use security type systems or dynamic taint tracking, both approximate in opposite directions.
- **'Can this data race occur?'** Undecidable, **which is why race detectors report both false positives (a race the program's logic prevents) and false negatives (a race on a path not exercised)** — and knowing which your detector is biased toward tells you how to use it.
- **'Will this smart contract terminate?'** **Which is why Ethereum charges gas — rather than deciding termination, make execution cost money and bound the budget, so non-termination becomes self-limiting by construction** (Module 06 §2's bounded escape). **The cleanest engineering response in this list**, and a good example of designing around a theorem rather than against it.
- **And 'is this set of signatures sufficient?'** — which is the same undecidability one level up, and is why no detection product can claim completeness.

BOUNDED MODEL CHECKING: GIVE UP THE QUANTIFIER ON PURPOSE

'Is there a bug?' is undecidable. **'Is there a bug reachable within k steps?'** is decidable — unroll the transition relation k times, encode it as a propositional formula, and hand it to a SAT solver (CSCE 625 Module 08 §3).

So the tool answers a strictly weaker question completely, rather than the real question partially.

Which is a different and better trade than it first appears: a complete answer to a stated weaker question is something you can reason about, and a partial answer to the real question is not.

And it is the dominant industrial verification technique for exactly that reason — **the answer is trustworthy within its bound, and the bound is explicit**. Hardware verification runs on this.

The honest claim is 'no bug within 20 steps', not 'no bug' — and a tool that reports the first while its users hear the second is the failure mode to watch for, because the tool is not wrong and the belief is.

Which is this program's scoped-claim discipline, in the one subject where the scope is a theorem rather than a measurement.

4 Reading a tool honestly

- **Which of sound, complete, and terminating did it give up? If the documentation does not say, assume it is neither sound nor complete** — which is the correct default and is the position most linters are in.
- **What is the bound?** Loop unrolling depth, call-graph depth, path count, timeout — **and what happens when the bound is hit. Silently giving up is very different from reporting that it gave up**, and the distinction is frequently undocumented.

- **What does 'no issues found' mean? 'Proved safe' and 'found nothing' are completely different claims, and tools report both with the same green tick. Find out which one yours means.**
- **What are the documented sources of unsoundness?** Reflection, dynamic class loading, native calls, concurrency, generated code — **most analysers simply do not model these and say so in a section nobody reads.**
- **And can you construct a false positive and a false negative? If you can, you understand the tool; if you cannot, you do not. This is the project, and it is the fastest way to calibrate how much to trust an analyser's output** — twenty minutes of adversarial construction tells you more than the manual.

WHAT TO TAKE FROM THIS MODULE

Undecidability is not abstract. It is why your tools behave the way they do, and the behaviour is derivable from the theorem rather than from the implementation — which means you can predict a tool's limitations before using it.

Every analyser gave up one of sound, complete, and terminating (§2), and which one it gave up determines what its output licenses you to believe.

Bounding a resource is the most successful response — gas, unrolling depth, timeouts, step limits — **because it converts an undecidable question into a decidable weaker one with an explicitly stated scope**, and a stated scope is what makes a claim usable.

And the tools are genuinely useful anyway, which is the point. Undecidability constrains what a tool can promise, not whether it is worth having — and Module 12 is about the engineering that lives in that gap.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Landi — Undecidability of static analysis (free)	§1's aliasing result, proved — may-alias is undecidable and so is may-alias for a restricted language.
Pierce — Types and Programming Languages	§2's decidability boundaries — which type systems are decidable and why the restrictions are where they are. Library copy; the exercises are free.
Cohen — Computer Viruses: Theory and Experiments (free)	§3's virus-detection undecidability , in the paper that introduced the term.
Clarke, Biere, Raimi & Zhu — Bounded Model Checking (free)	§3's method — and the clearest statement of what a bounded result does and does not establish.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write code where a compiler must assume aliasing** and inspect the generated assembly.
- 2 **Add restrict or equivalent** and compare. Report the difference.
- 3 **Find a dead branch your compiler does not eliminate** and explain why.
- 4 **Write a program your type checker rejects that would have worked**, and identify the restriction responsible.
- 5 **Classify five tools you use** as sound, complete, terminating — two of three each.
- 6 **Construct a false positive** for one of them, with code.
- 7 **Construct a false negative** for the same tool.
- 8 **Read that tool's documentation** for its unsoundness sources and report what it admits.
- 9 **Run a bounded model checker** with two different bounds and report what changes.
- 10 **Write the honest claim** its output supports, in one sentence.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name six undecidable problems a compiler faces and what it does about each.
- 2 Why is aliasing the most expensive one, and what are three responses to it?
- 3 Why is type checking decidable, and what does that cost?
- 4 State the pick-two-of-three frame and give five tools' positions.
- 5 Name four undecidable problems in security or verification.
- 6 Why does Ethereum charge gas, in this module's terms?
- 7 What does bounded model checking give up, and what does it gain?
- 8 Give five questions to ask of an analyser.
- 9 What is the fastest way to calibrate a tool?